

Collider Event System

Documentation

Version 0.4.4

Component-based collider/condition-driven event system for Unity: attach ordered Condition and Action components to a GameObject to build zone triggers and always-on condition watchers — without writing code.

Table of Contents

1. Introduction	3
2. Installation	3
3. Quick Start	3
4. Core Components	3
4.1 Collider Event	3
4.2 Condition Watcher	4
5. Conditions	4
6. Actions	4
7. Variables	4
8. Base Options	4
8.1 Hold Time	4
8.2 After Trigger	4
8.3 Hint Material	5
9. Ready-Made Prefabs	5
10. Demo Sample	5
11. Known Gotchas	5
12. Credits & Support	6

1. Introduction

Collider Event System lets you build zone triggers and always-on condition checks entirely from the Inspector, by attaching ordered **Condition** and **Action** components to a GameObject. No scripting is required to use any of the built-in Conditions or Actions.

This guide covers installation, the core components, every built-in Condition and Action, Variables, and the options shared by every trigger.

2. Installation

Collider Event System is distributed as a Unity Package (UPM). After adding it to your project, it appears under **Packages / Collider Event System** in the Project window, alongside your other installed packages.

To add the included sample scene:

- 1 Open **Window > Package Manager**.
- 2 Select **Collider Event System** in the package list.
- 3 Open the **Samples** tab and click **Import** next to **Demo**.

The Demo sample's materials are authored for URP (Universal Render Pipeline). On a project still using the Built-in Render Pipeline, they will appear pink until you run **Window > Rendering > Render Pipeline Converter** (Unity's own built-in fixer) — a one-time, couple-clicks step, not something wrong with the package itself. The package's actual Conditions and Actions have no material or shader dependency at all; this only affects the optional Demo sample.

3. Quick Start

- 1 Drag one of the ready-made prefabs (**Trigger Cube**, **Trigger Sphere**, or **Trigger Capsule** — in this package's **Prefabs** folder) into your scene.
- 2 Under **Conditions**, click **Add Condition** and pick one (e.g. Distance).
- 3 Under **Actions**, click **Add Action** and pick one (e.g. Invoke Events).
- 4 Enter Play Mode and walk into the zone.

To add a trigger to a GameObject of your own instead: add a Collider (e.g. Box Collider), tick **Is Trigger**, then add the **Collider Event** component yourself.

4. Core Components

4.1 Collider Event

Add alongside any native Collider (marked **Is Trigger**). Waits for something to enter the zone, then checks the Conditions. You can optionally restrict which Tags can trigger it — leave every box unchecked to let anything with a Collider trigger it.

4.2 Condition Watcher

No Collider needed. Checks the Conditions continuously from the moment the scene starts — for things that don't happen at a place (e.g. "wait until the player has 3 keys").

5. Conditions

Available Conditions: **Looking At, Input, Variable, Distance, Rotation.**

Each Condition after the first has an **And / Or** selector relating it to the one above it, read strictly top to bottom — e.g. "A Or B And C" reads as "(A Or B) And C".

6. Actions

Available Actions: **Animation, Audio, Scene, GameObject, Material, Rigidbody, Transform, Variable, Invoke Events, Instantiate Prefab.**

7. Variables

Instead of typing a string key, create a Variable asset (**Assets > Create > Collider Event System > Variables**) and drag it into a Variable Condition or Variable Action — or click the **+** next to the field to create one on the spot. Toggle **Persistent** on a Variable to have it saved to disk and reloaded automatically between play sessions.

A Variable asset is shared — every GameObject that references the same one reads/writes the same value. There is currently no built-in way to give each instance of a prefab (e.g. each enemy) its own independent value like HP without writing a script for it.

8. Base Options

8.1 Hold Time

How many seconds the Conditions must stay true, without interruption, before Actions run.

8.2 After Trigger

What happens once Actions have run: deactivate the GameObject, destroy it, destroy its parent, do nothing and allow it to reset, or wait and run Exit Actions once the Conditions/zone become inactive again.

8.3 Hint Material

Optional. A material applied to a target while the Conditions above aren't all met yet, and automatically restored once they are (or once the zone is left before they are) — useful as a hint that an interaction is available, e.g. highlighting an object until the player actually interacts with it. It reflects the combined Condition list result (respecting And/Or), not any single Condition on its own.

9. Ready-Made Prefabs

The package's **Prefabs** folder has ready-made trigger zones (**Trigger Cube**, **Trigger Sphere**, **Trigger Capsule**) — each a Collider set to **Is Trigger** with a Collider Event already attached, ready to drag into a scene as shown in Quick Start.

10. Demo Sample

Import **Demo** from this package's page in the Package Manager window (see Section 2) for a walkable scene that puts several Conditions, Actions, and Variables to use together.

11. Known Gotchas

A few things that aren't bugs, but are easy to trip over:

- **Rotation Condition measures world rotation by default.** If the object you're checking has a rotated parent, its world rotation includes the parent's rotation too — not just the number shown on the object's own Transform. Switch **Space** to **Self** to compare its local rotation instead.
- **Transform.eulerAngles is always 0-360, never negative.** A slight rotation "the wrong way" (e.g. -5°) reads as 355°, which can satisfy a "Greater Than" comparison you didn't intend. Rotation Condition already normalizes this internally, but keep it in mind if you inspect rotation values elsewhere.
- **Scene Action loading a scene not in Build Settings works in the Editor, but not in a real build.** Unity's Editor Play Mode has a convenience fallback that resolves a scene by name across the whole project; a built player doesn't have it. Add every scene you load to Build Settings before shipping.
- **Cancel Physics (Transform Action) doesn't pause momentum, it erases it.** A kinematic Rigidbody always reports zero velocity. Once restored to non-kinematic after the move finishes, the object resumes at rest, not from whatever speed it had before — it won't carry on falling at its old speed, for example.
- **Drag from the Hierarchy, not the Project window, into a Target field.** Dragging a prefab asset gives you a reference to the asset itself (whose values never change at runtime), not the instance in your scene. This is a general Unity gotcha, not specific to this package, but it's an easy mix-up.
- **A Cylinder's Collider is actually a Capsule.** Unity has no native cylinder-shaped collider — the default Cylinder primitive gets a Capsule Collider, rounded ends included. The Debug Color gizmo draws the real collider shape, so a "cylinder" zone will show (and behave) like a capsule.

12. Credits & Support

Initially built starting from the concept behind Alexander Scott's Enhanced Trigger Box — a trigger volume driven by composable conditions and actions. This package has since diverged into its own architecture (Conditions, Actions, and Variables as separate composable components, a Condition Watcher for non-physical triggers, custom Inspector tooling, etc.), but the original idea is worth crediting.

For the full version history, see the CHANGELOG included with the package.